is associated with the current method. Using this method, you could implement something similar to the standard File.open (again, ignoring error handling) using the following:

```
class File
  def self.my_open(*args)
    file = File.new(*args)
    return file unless block_given?
    result = yield file
    file.close
    result
  end
end
```

In this version, we use the guard clause return file unless block_given? to exit the method early if block_given? is false. Otherwise, we proceed with the same yield then close as our previous code.

This code has one last missing piece: in the previous examples of using blocks to control resources, we didn't address error handling. If we wanted to implement these methods properly, we'd need to ensure that we closed a file even if the code processing that file somehow aborted. We do this using exception handling, which we talk about later on page 171.

## Using Blocks as Objects

Blocks are like anonymous methods, but there's more to them than that. You can also store a block in a variable, pass it as an argument to a function, and then invoke its code later.

Remember we said that you can think of blocks as an extra implicit argument that's passed to a method? Well, you can make that argument explicit. If the last parameter in a method definition is prefixed with an ampersand (such as &action), Ruby looks for a code block whenever that method is called. That code block is converted to an object of class Proc and assigned to the parameter. You can then treat the parameter as any other variable.

Here's an example where we create a Proc object in one instance method and store it in an instance variable. We then invoke the proc from a second instance method.

```
class ProcExample
  def pass_in_block(&action)
    @stored_proc = action
  end

  def use_proc(parameter)
    @stored_proc.call(parameter)
  end
end

eg = ProcExample.new
eg.pass_in_block { |param| puts "The parameter is #{param}" }
eg.use_proc(99)
```

*produces:*

```
The parameter is 99
```

See how the call method on a proc object invokes the code in the original block?